

Snake Game Project

Software Design Documentation

Table of Contents

1. Introduction	2
2. System Requirements Specification (SRS)	3
3. Design Architecture & Class Diagram	4
4. Implementation Logic (The Game Loop)	6
5. Conclusion	7

1. Introduction

This project is a classic implementation of the **Snake Game** using the **C++** programming language. The core objective of the project is to demonstrate the application of **Object-Oriented Programming (OOP)** principles in game development. By encapsulating game entities like the snake and food into distinct classes, the system achieves modularity, maintainability, and clear logic separation. The game operates in a console environment, utilizing standard input/output libraries and Windows-specific headers for real-time interaction.

2. System Requirements Specification (SRS)

2.1 Functional Requirements

- **Character Control:** The system must interpret user keystrokes ('w', 'a', 's', 'd') to dictate the direction of the snake.
- **Continuous Movement:** The snake must move autonomously in the last selected direction until a collision occurs or a new direction is provided.
- **Food Generation:** A food object ('F') must appear at a randomized coordinate within the grid after the previous one is consumed.
- **Score and Growth:** Upon collision between the snake's head and the food, the player's score increases by 10 points and the snake's tail length increments by one.
- **Termination Conditions:** The game state must change to "Game Over" if the snake hits

the grid boundaries or intersects its own body.

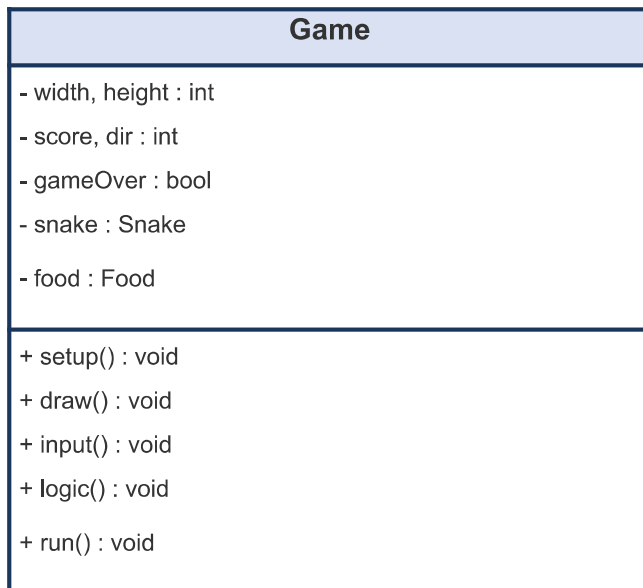
2.2 Non-Functional Requirements

- **Usability:** The interface must be clear, using character representations for the environment.
- **Reliability:** The game must consistently handle rapid input changes without crashing.

3. Design Architecture

The system follows a modular architecture where the game controller manages individual entities. This ensures that the data of the snake and food is protected and only modified through controlled methods.

3.1 UML Class Diagram



Snake
- x, y : int - tailX, tailY : int[100] - length : int
+ move(int) + grow() + updateTail()

Food
- x, y : int
+ generate(int, int) + getX(), getY()

3.2 Component Breakdown

- **Class Snake:** Responsible for maintaining the position of the head and all segments of the tail. It contains the logic for shifting the tail coordinates to follow the head.
- **Class Food:** A utility class that uses the rand() function to determine its placement within the valid game space.
- **Class Game:** The primary engine that initializes the variables, handles the console output (rendering), and implements the core win/loss logic.

4. Implementation Logic (The Game Loop)

The program execution logic is centralized in the Game::run() method, which implements a infinite loop that only breaks when the gameOver flag is raised.

Phase	Detailed Description
1. Draw	Clears the console using coordinate manipulation (SetConsoleCursorPosition) and prints the grid boundaries, snake, and food symbols.
2. Input	Uses _kbhit() to check for a key press without pausing the game. If a key is pressed, _getch() captures the character to update the direction.
3. Logic	The snake moves. The system checks if the new head position overlaps with a wall, the tail, or the food. If food is eaten, the score and length are incremented.
4. Sleep	A 120ms delay is introduced to make the game playable and synchronize the frame rate.

5. Conclusion

The Snake Game project successfully demonstrates the power of Object-Oriented Programming in C++. By separating the entities into classes, the code remains clean and easy to expand. Future enhancements could include adding multiple levels, power-ups, or a graphical user interface (GUI). This documentation serves as a complete blueprint of the current system design and implementation.